

ECE 552 / CPS 550

Advanced Computer Architecture I

Lecture 2

CISC and Microcoding

Benjamin Lee
Electrical and Computer Engineering
Duke University

www.duke.edu/~bcl15
www.duke.edu/~bcl15/class/class_ece252fall12.html



Microarchitectures

Instruction Set Architecture (ISA)

- Defines the hardware-software interface
- Provides convenient functionality to higher levels (e.g., software)
- Provides efficient implementation to lower levels (e.g., hardware)

Microarchitecture

- Microarchitecture implements ISA
- Implementation abstracted from programmer

Early Microarchitectures

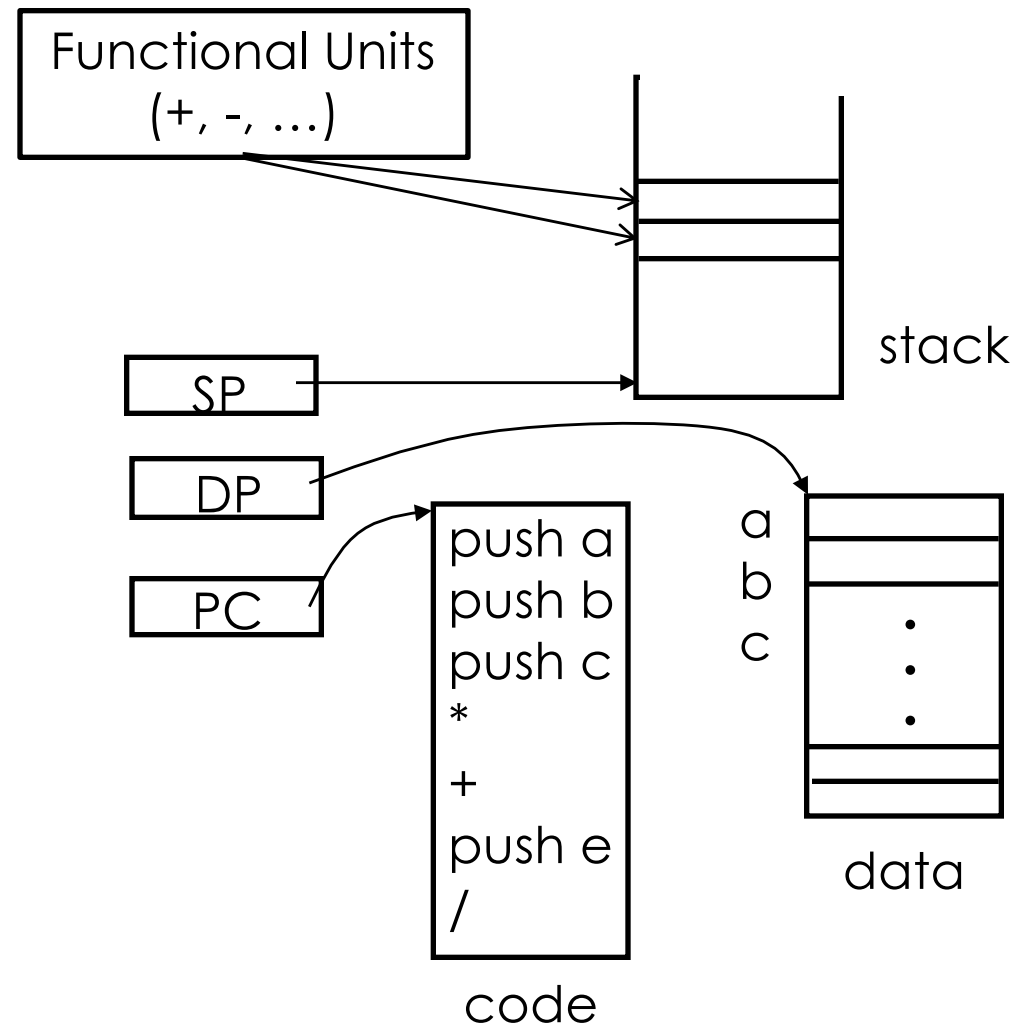
- Stack Machines (1960s)
- Microprogrammed Machines (1970s-1980s)
- Reduced Instruction Set Computing (1990s)

Stack Machines



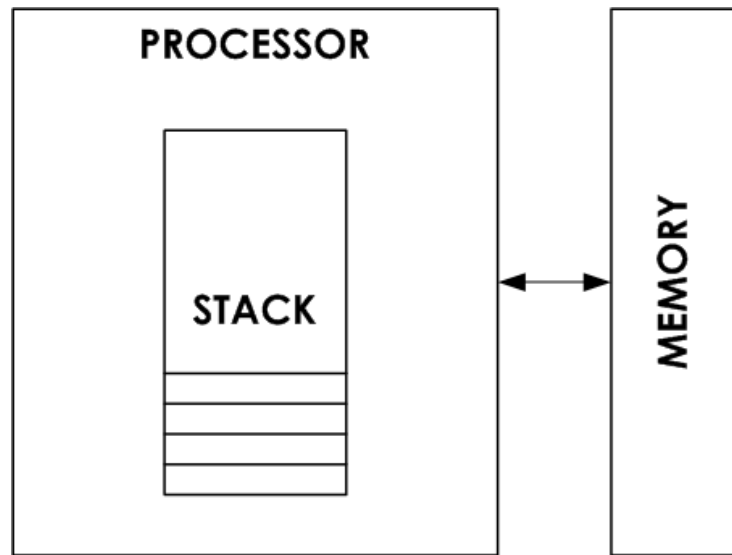
Stack Machine Overview

- Stack operations (e.g., push/pop)
- Computation (e.g., +, -, ...)
- **Data pointer (DP)** to access element in data area
- **Program counter (PC)** to access instruction in code area
- **Stack pointer (SP)** to access, move element in stack frame





Stack Machine Overview

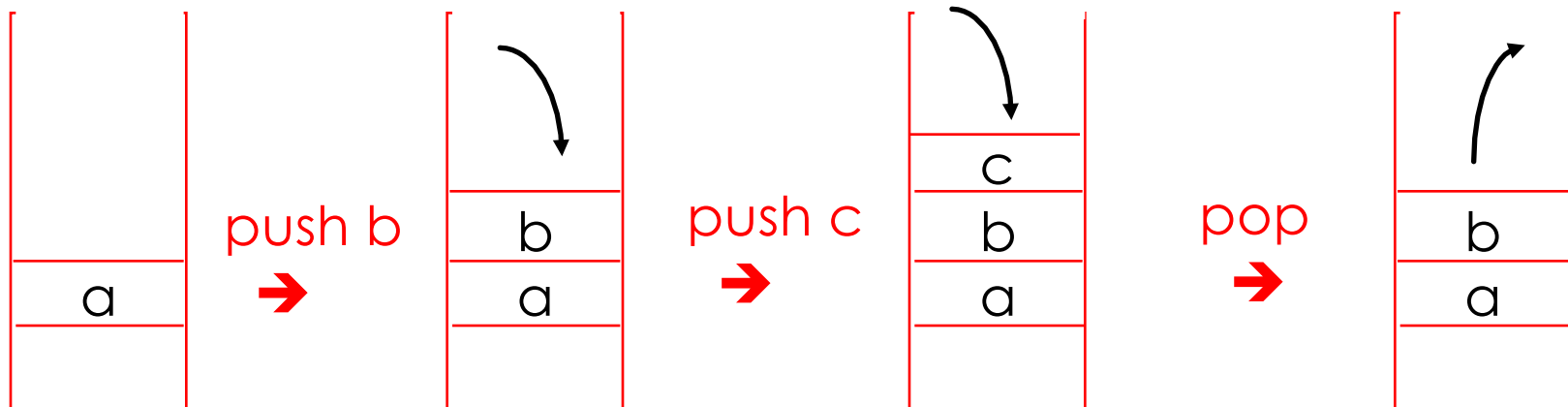


Processor state includes stack

typical operations:

push, pop, +, *, ...

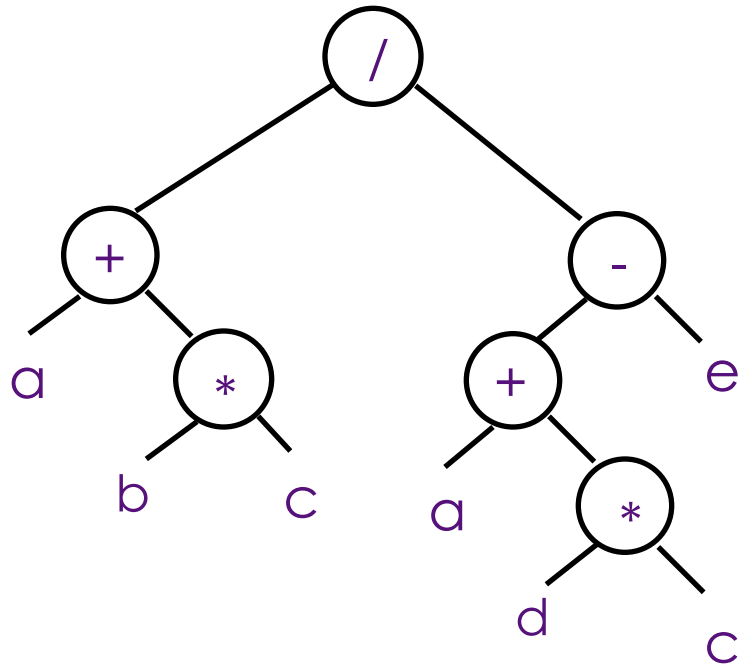
Instructions, like +, implicitly specify top 2 stack elements as operands.





Expression Evaluation

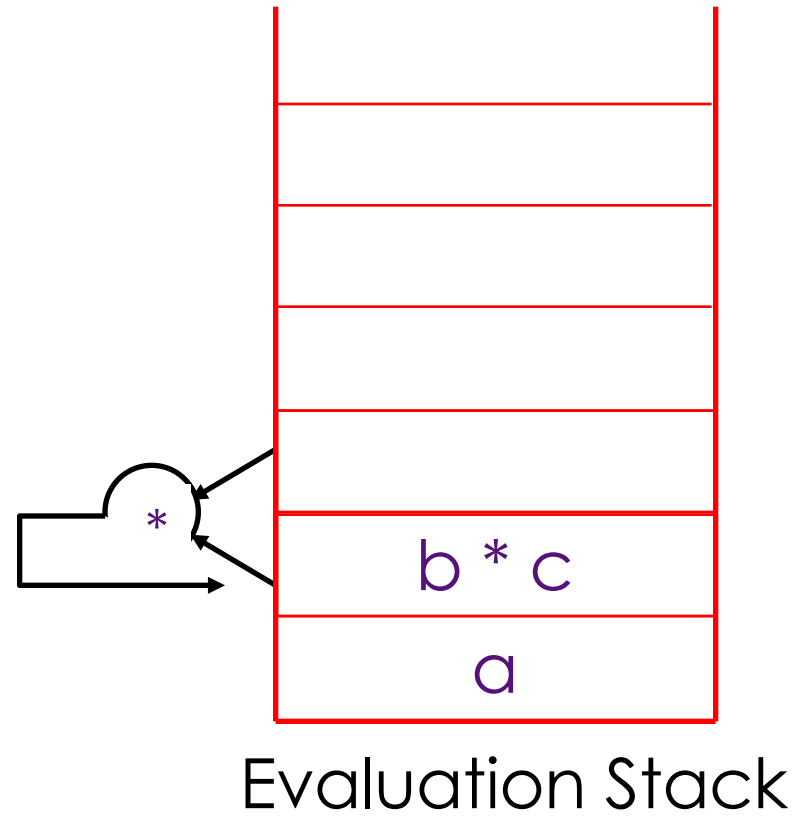
$$(a + b * c) / (a + d * c - e)$$



Reverse Polish

a b c * + a d c * + e - /

↑ ↑ ↑ ↑
push push push multiply





Stack Hardware Organization

Stack as Processor State

- Processor state includes part of stack
- Processor state is bounded, small → tens of stack elements in registers
- Remainder of stack in main memory

Stacks and Memory References

- Option 1: Top 2 stack elements in registers, remainder in memory
- Each push/pop requires memory reference; poor performance
- Option 2: Top N stack elements in registers, remainder in memory
- Overflows/underflows require memory reference; better performance
- Analogous to register spilling



Stack Size & Memory References

$$(a+b*c)/(a+d*c-e) \rightarrow a\ b\ c\ * + a\ d\ c\ * + e\ - /$$

	<i>program stack (size = 2)</i>	<i>memory refs</i>
push a	R0	a
push b	R0 R1	b
push c	R0 R1 R2	c, ss(a)
*	R0 R1	sf(a)
+	R0	
push a	R0 R1	a
push d	R0 R1 R2	d, ss(a+b*c)
push c	R0 R1 R2 R3	c, ss(a)
*	R0 R1 R2	sf(a)
+	R0 R1	sf(a+b*c)
push e	R0 R1 R2	e,ss(a+b*c)
-	R0 R1	sf(a+b*c)
/	R0	

- 4 stack stores (ss), 4 stack fetches (sf)
- Implicit memory references when program stack exceeds processor capacity



Stack Size & Evaluating Expressions

$(a+b*c)/(a+d*c-e) \rightarrow a\ b\ c\ *\ +\ a\ d\ c\ *\ +\ e\ -\ /\$

	<i>program stack (size = 4)</i>
push a	R0
push b	R0 R1
push c	R0 R1 R2
*	R0 R1
+	R0
push a	R0 R1
push d	R0 R1 R2
push c	R0 R1 R2 R3
*	R0 R1 R2
+	R0 R1
push e	R0 R1 R2
-	R0 R1
/	R0

- Note a and c are loaded twice $\rightarrow$ inefficient register use



vs General Purpose Register File

$$(a+b*c)/(a+d*c-e) \rightarrow a \ b \ c \ * \ + \ a \ d \ c \ * \ + \ e \ - \ /$$

Load	R0	a	
Load	R1	c	
Load	R2	b	
Mul	R2	R1	Reuse R2, Store result into R2
Add	R2	R0	Reuse R2, ...
Load	R3	d	
Mul	R3	R1	Reuse R3, Store result into R3
Add	R3	R0	Reuse R3, ...
Load	R0	e	Reuse R0, ...
Sub	R3	R0	Reuse R3, ...
Div	R2	R3	Reuse R2, ...

- Use registers efficiently with explicit names
- Reduce unnecessary loads, stores
- Require fewer registers but longer instructions



vs General Purpose Registers

- Amdahl, Blaauw, and Brooks. "Architecture of the IBM System/360." 1964

"In the final analysis, the stack organization would have been about break-even...the general-purpose objective weighed heavily in favor of the more flexible addressed register organization."

1. Stack machine's advantage is from fast registers, not how they're used
2. Surfacing instructions, which bring submerged data to active positions, is profitable 50% of the time due to repeated operands.
3. Code density for stacks, register files is comparable
4. Stack depth limited by number of fast registers; requires stack management
5. Stacks benefit recursive sub-routines but requires independently addressed stacks (SP management)
6. Fitting variable-length fields into fixed-width stack is awkward



Transition from Stack Machines

Stack machine's popularity faded in the 1980s

Code Density

- Stack programs are not necessarily smaller
- Consider frequent stack surfacing instructions
- Consider short register addresses

Advent of Modern Compilers

- Stack machines require managing finite capacity
- Register allocation improves register use
- Early language-directed architectures did not account for compilers

Microprogrammed Machines



Microprogrammed Machines

Why do we care?

- Illustrate small processors with complex instruction sets (CISC)
- Understand part of modern machines (x86, PowerPC, IBM360)

ISA favors particular microarchitecture style

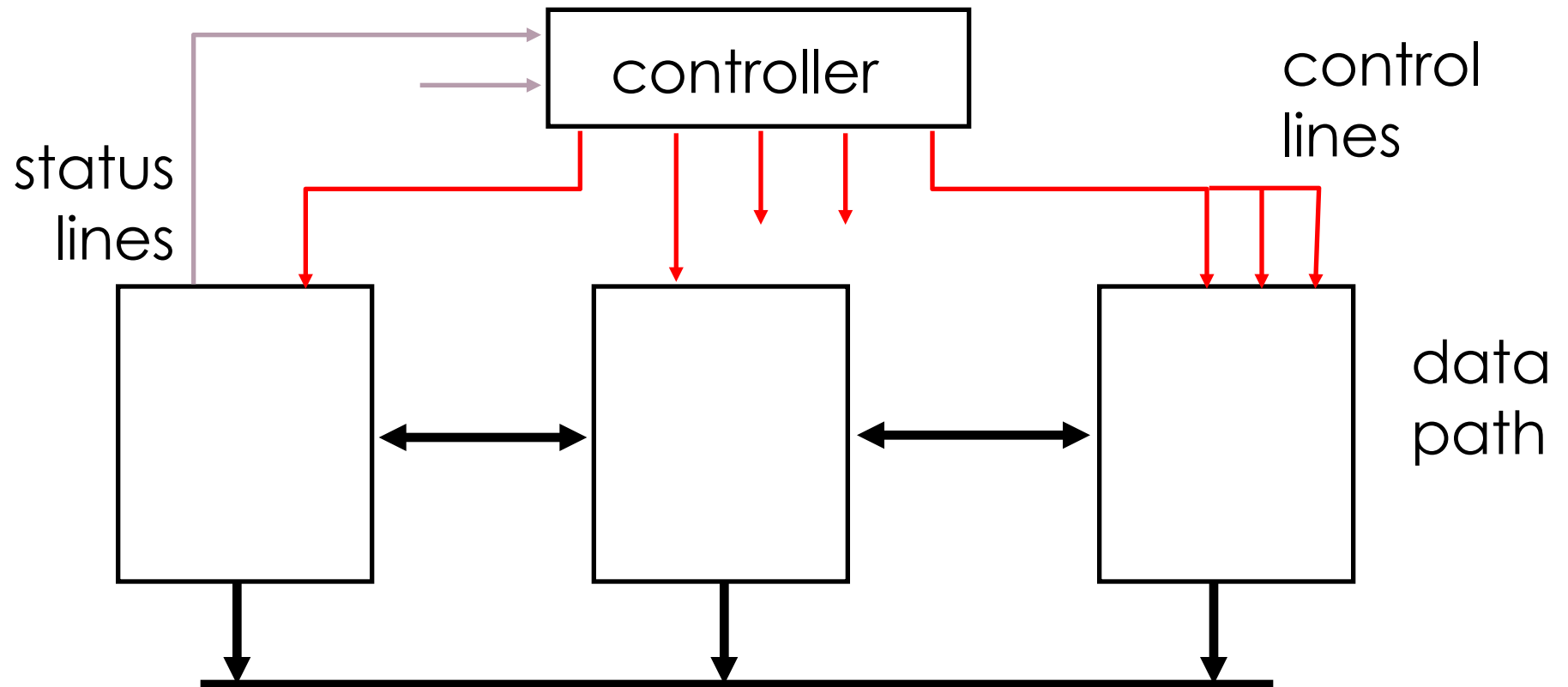
- Complex Instruction Set Computer (CISC) → microcoded
- Reduced Instruction Set Computer (RISC) → hardwired, pipelined
- Very Long Instruction Word (VLIW) → fixed latency, in-order pipelines
- Java Virtual Machine (JVM) → software interpretation

ISA can use any microarchitecture style

- Core2 Duo: CISC (x86) with hardwired pipeline and microcode
- This Lecture: RISC (MIPS) with microcode



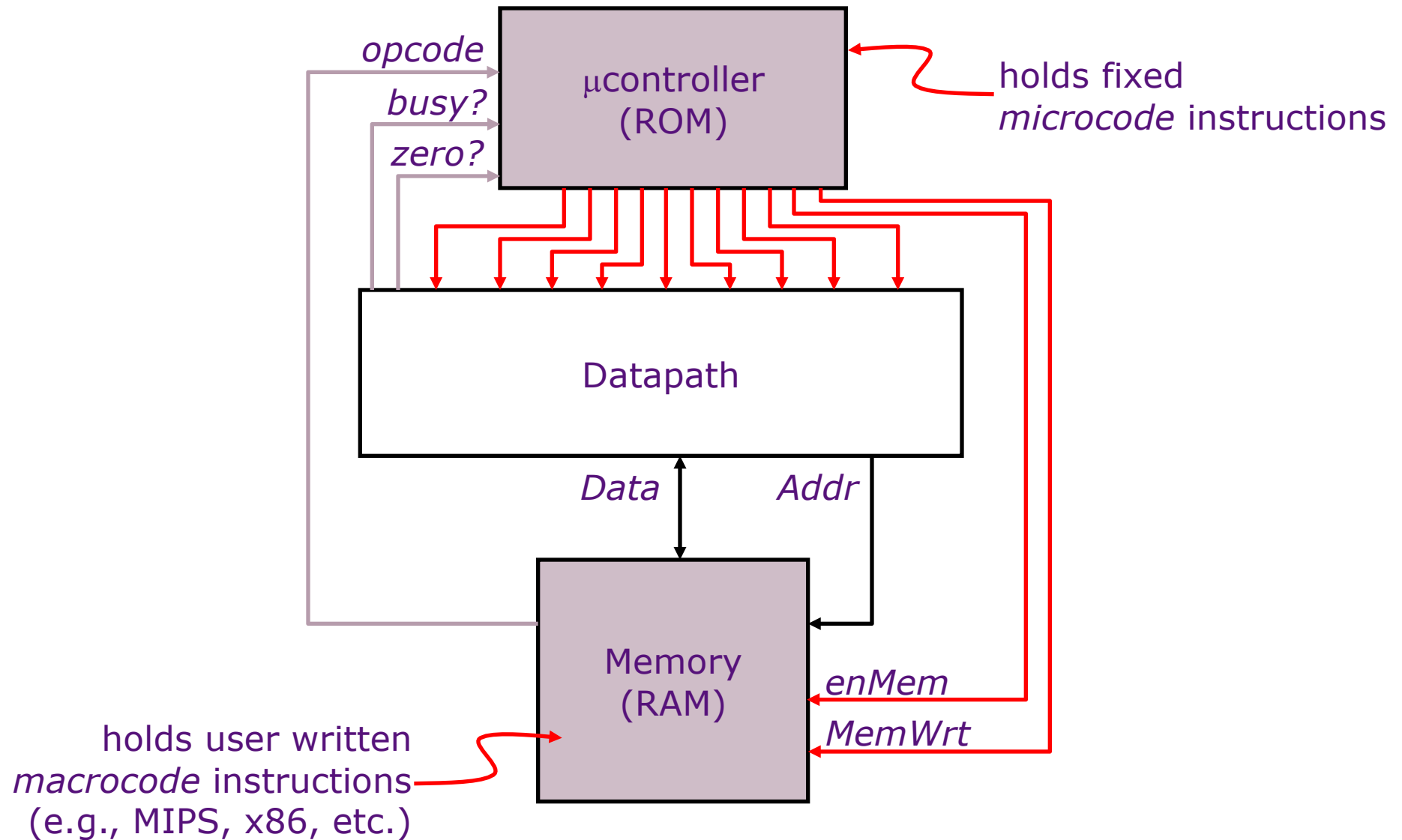
Hardware Organization



- Structure → How are components connected? Statically
- Behavior → How does data move between components? Dynamically



Microcoded Microarchitecture





MIPS32 ISA

See Hennessy and Patterson, Appendix for full description

Processor State

- 32 32-bit GPRs, R0 always contains a 0
- 16 double-precision, 32 single-precision FPRs
- FP status register, used for FP compares & exceptions
- PC, the program counter
- Other special registers

Data Types

- 8-bit byte, 16-bit half word
- 32-bit word for integers
- 32-bit word for single-precision floating-point
- 64-bit word for double-precision floating-point

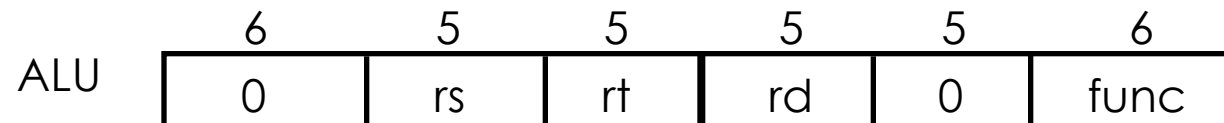
Load/Store ISA

- Data addressing modes: immediate & indexed
- Branch addressing modes: PC relative & register indirect
- Byte addressable memory: big-endian mode

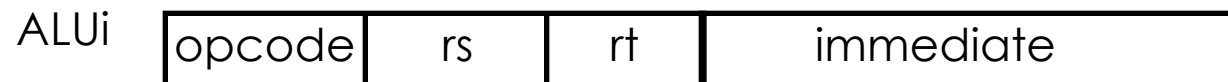
Instructions are 32 bits



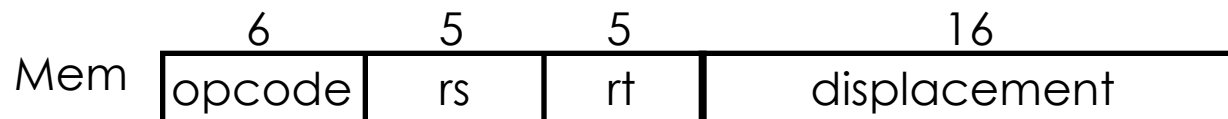
MIPS Instruction Formats



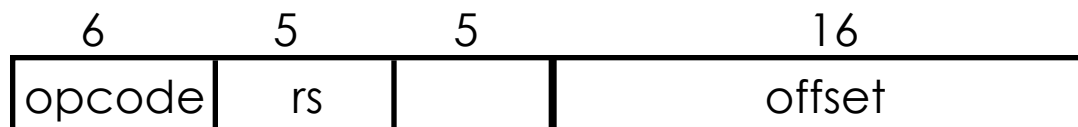
$(rd) \leftarrow (rs) \text{ func } (rt)$



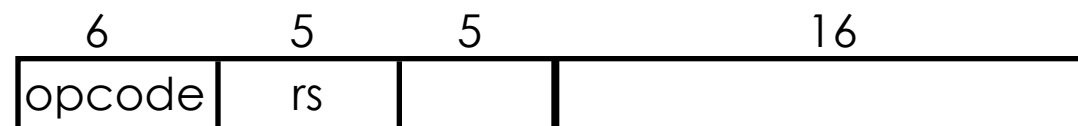
$(rt) \leftarrow (rs) \text{ op } \text{immediate}$



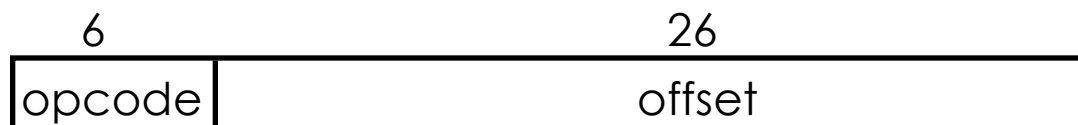
$M[(rs) + \text{displacement}]$



BEQZ, BNEZ



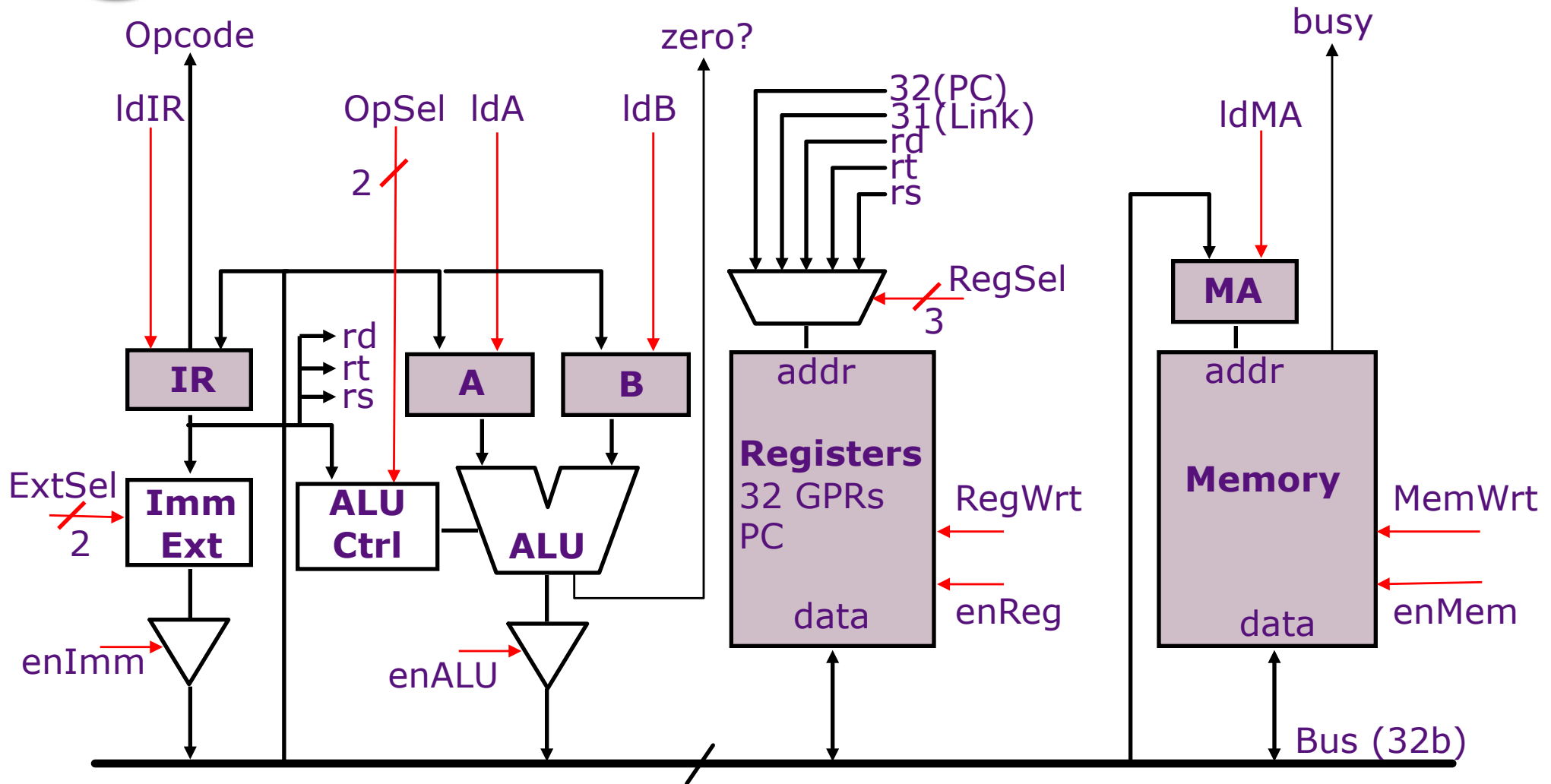
JR, JALR



J, JAL



Bus-Based MIPS Datapath



Microinstruction specifies register to register transfer (17 control signals)

PC → MA means RegSel=PC; enReg=yes; IdMA=yes

Reg[rt] → B means RegSel=rt; enReg=yes; IdB=yes



Executing Instructions

1. Fetch instruction from memory @ PC
2. Decode register
3. Perform ALU operation
4. Access memory (optional)
5. Write register (optional)
+ compute next PC



Macroinstruction $\rightarrow$ Microprogram

LW:	$A \leftarrow \text{Reg}[\text{rs}]$ $B \leftarrow \text{Imm}$ $\text{MA} \leftarrow A + B$ $\text{Reg}[\text{rt}] \leftarrow \text{Memory}$ do instruction fetch	# compute address # load from memory
J:	$A \leftarrow \text{PC}$ $B \leftarrow \text{IR}$ $\text{PC} \leftarrow \text{JumpTarg}(A,B)$ do instruction fetch	# $\text{JumpTarg}(A,B) = \{A[31:28], B[25:0], 00\}$
beqz:	$A \leftarrow \text{Reg}[\text{rs}]$ <i>If zero?(A) then go to bz-taken</i> do instruction fetch	
bz-taken:	$A \leftarrow \text{PC}$ $B \leftarrow \text{Imm} \ll 2$ $\text{PC} \leftarrow A + B$ do instruction fetch	



Microprogram in the ROM

State	Op	zero?	busy	Control points	next-state
fetch ₀	*	*	*	MA $\leftarrow$ PC	fetch ₁
fetch ₁	*	*	yes		fetch ₁
fetch ₁	*	*	no	IR $\leftarrow$ Memory	fetch ₂
fetch ₂	*	*	*	A $\leftarrow$ PC	fetch ₃
fetch ₃	*	*	*	PC $\leftarrow$ A + 4	?
fetch ₃	ALU	*	*	PC $\leftarrow$ A + 4	ALU ₀
ALU ₀	*	*	*	A $\leftarrow$ Reg[rs]	ALU ₁
ALU ₁	*	*	*	B $\leftarrow$ Reg[rt]	ALU ₂
ALU ₂	*	*	*	Reg[rd] $\leftarrow$ func(A,B)	fetch ₀



Microprogram in the ROM

State	Op	zero?	busy	Control points	next-state
fetch ₀	*	*	*	MA ← PC	fetch ₁
fetch ₁	*	*	yes		fetch ₁
fetch ₁	*	*	no	IR ← Memory	fetch ₂
fetch ₂	*	*	*	A ← PC	fetch ₃
fetch ₃	ALU	*	*	PC ← A + 4	ALU ₀
fetch ₃	ALUi	*	*	PC ← A + 4	ALUi ₀
fetch ₃	LW	*	*	PC ← A + 4	LW ₀
fetch ₃	SW	*	*	PC ← A + 4	SW ₀
fetch ₃	J	*	*	PC ← A + 4	J ₀
fetch ₃	JAL	*	*	PC ← A + 4	JAL ₀
fetch ₃	JR	*	*	PC ← A + 4	JR ₀
fetch ₃	JALR	*	*	PC ← A + 4	JALR ₀
fetch ₃	beqz	*	*	PC ← A + 4	beqz ₀
...					
ALU ₀	*	*	*	A ← Reg[rs]	ALU ₁
ALU ₁	*	*	*	B ← Reg[rt]	ALU ₂
ALU ₂	*	*	*	Reg[rd] ← func(A,B)	fetch ₀



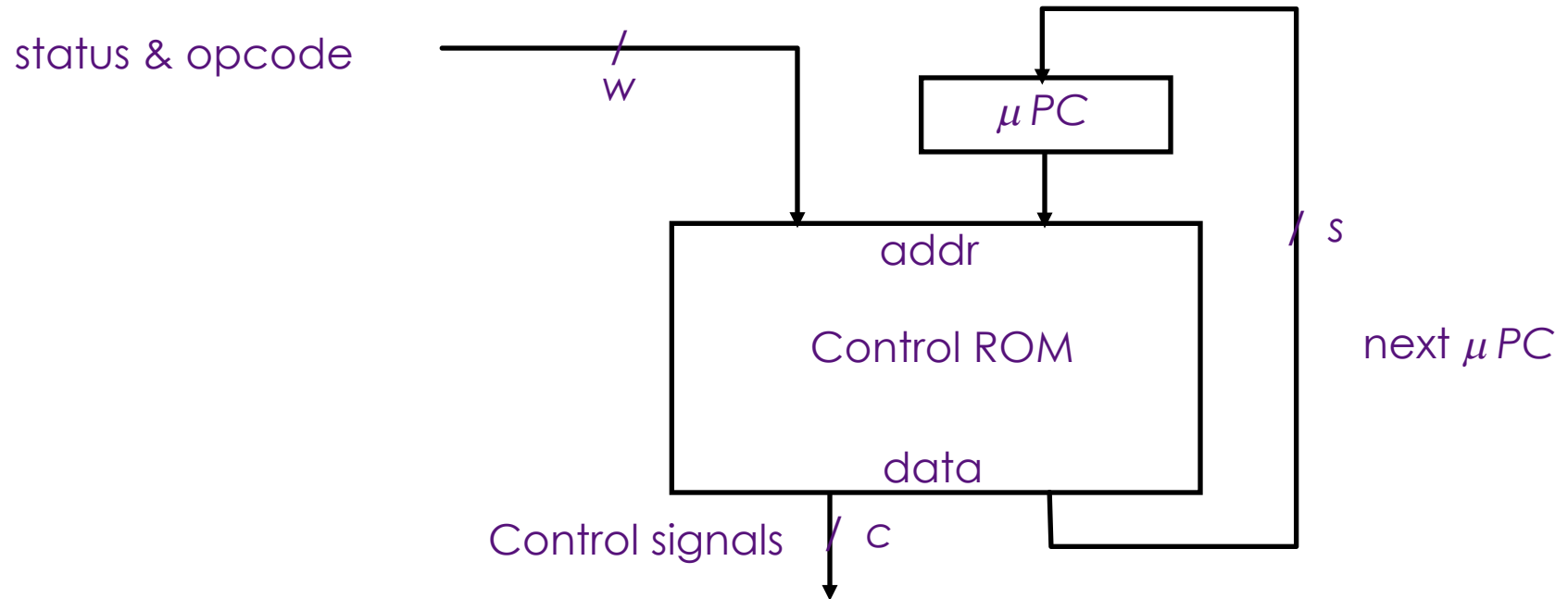
Microprogram in the ROM

State	Op	zero?	busy	Control points	next-state
ALUi ₀	*	*	*	$A \leftarrow \text{Reg}[\text{rs}]$	ALUi ₁
ALUi ₁	sExt	*	*	$B \leftarrow \text{sExt}_{16}(\text{Imm})$	ALUi ₂
ALUi ₁	uExt	*	*	$B \leftarrow \text{uExt}_{16}(\text{Imm})$	ALUi ₂
ALUi ₂	*	*	*	$\text{Reg}[\text{rd}] \leftarrow \text{Op}(A, B)$	fetch ₀
...					
J ₀	*	*	*	$A \leftarrow \text{PC}$	J ₁
J ₁	*	*	*	$B \leftarrow \text{IR}$	J ₂
J ₂	*	*	*	$\text{PC} \leftarrow \text{JumpTarg}(A, B)$	fetch ₀
...					
beqz ₀	*	*	*	$A \leftarrow \text{Reg}[\text{rs}]$	beqz ₁
beqz ₁	*	yes	*	$A \leftarrow \text{PC}$	beqz ₂
beqz ₁	*	no	*		fetch ₀
beqz ₂	*	*	*	$B \leftarrow \text{sExt}_{16}(\text{Imm})$	beqz ₃
beqz ₃	*	*	*	$\text{PC} \leftarrow A + B$	fetch ₀
...					

$\text{JumpTarg}(A, B) = \{A[31:28], B[25:0], 00\}$



Size of Control Store



$$\text{size} = 2^{(w+s)} \times (c + s)$$

$$w = 6 \text{ (opcode)} + 2 \text{ (status)}, c = 17 \text{ (signals)}, s = ?$$

steps per opcode = 4 to 6 + fetch-sequence

states = (4 steps per op-group) $\times$ op-groups + shared microprograms

$$= 4 \times 8 + 10 = 42$$

$$\rightarrow s = 6$$

$$\text{Control ROM} = 2^{(8+6)} \times 23 \text{ bits} = 48 \text{ Kbytes}$$



Reducing Size of Control Store

Control store must be fast, which is expensive

Reduce ROM height (= address bits)

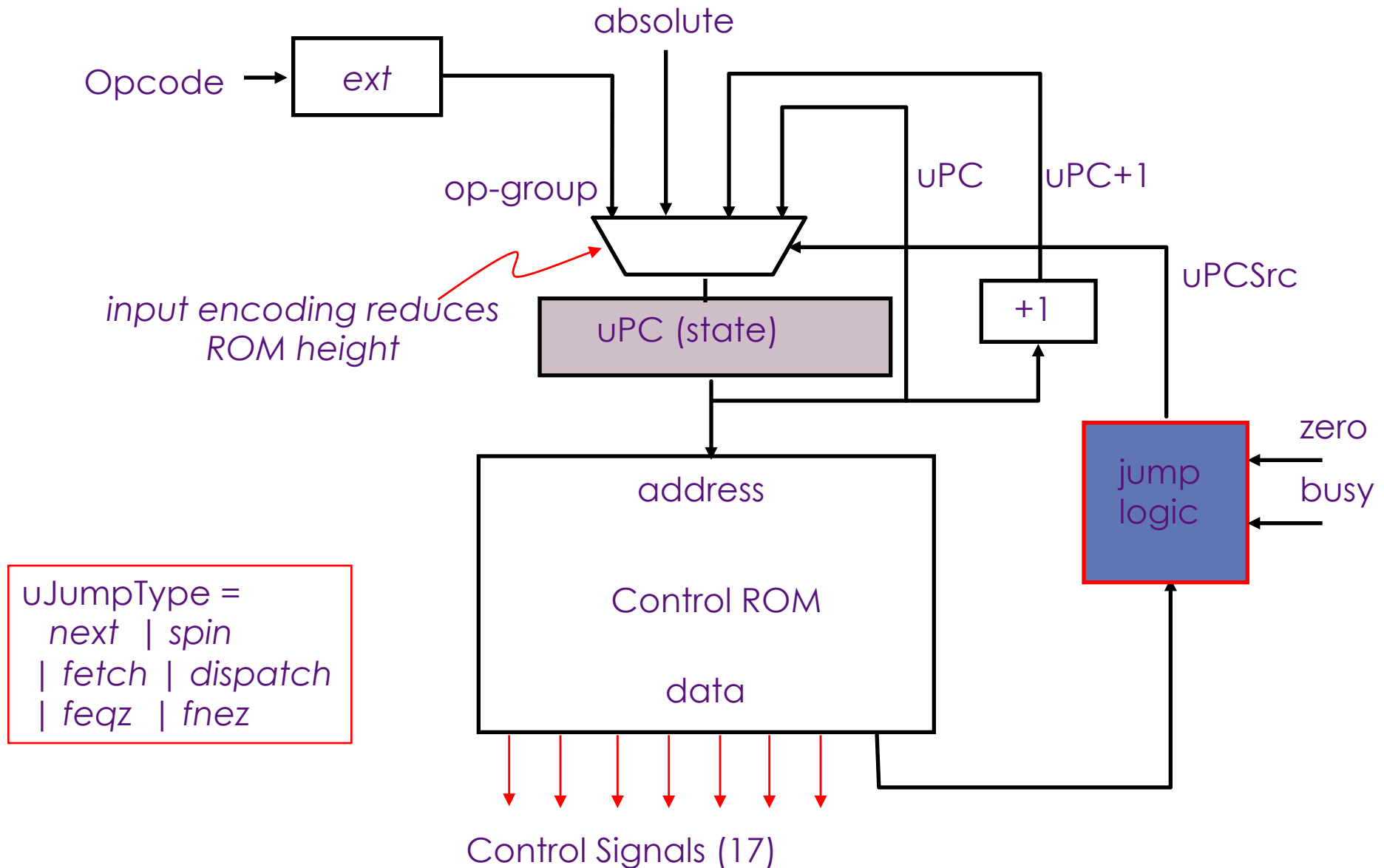
- reduce inputs by extra external logic
- each input bit doubles the size of the control store
- reduce states by grouping opcodes
- find common sequences of actions
- condense input status bits
- combine all exceptions into one, i.e., exception/no-exception

Reduce ROM width

- restrict the next-state encoding
- next, dispatch on opcode, wait for memory, ...
- encode control signals (vertical microcode)



MIPS Microcontroller v2





Jump Logic

$\mu\text{PCSrc} = \text{Case } \mu\text{JumpTypes}$

next $\rightarrow \mu\text{PC}+1$

spin $\rightarrow$ if (busy) then μPC else $\mu\text{PC}+1$

fetch $\rightarrow$ absolute

dispatch $\rightarrow$ op-group

feqz $\rightarrow$ if (zero) then absolute else $\mu\text{PC}+1$

fnez $\rightarrow$ if (zero) then $\mu\text{PC}+1$ else absolute



Instruction Fetch and ALU

State	Control Points	Next State
fetch0	$MA \leftarrow PC$	next
fetch1	$IR \leftarrow \text{Memory}$	spin
fetch2	$A \leftarrow PC$	next
fetch3	$PC \leftarrow A+4$	dispatch
...		
ALU0	$A \leftarrow \text{Reg}[rs]$	next
ALU1	$B \leftarrow \text{Reg}[rt]$	next
ALU2	$\text{Reg}[rd] \leftarrow \text{func}(A,B)$	fetch
...		
ALUi0	$A \leftarrow \text{Reg}[rs]$	next
ALUi1	$B \leftarrow \text{sExt16}(\text{Imm})$	next
ALUi2	$\text{Reg}[rd] \leftarrow \text{Op}(A,B)$	fetch



Load and Store

State	Control Points	Next State
LW0	$A \leftarrow \text{Reg}[\text{rs}]$	next
LW1	$B \leftarrow \text{sExt16}(\text{Imm})$	next
LW2	$MA \leftarrow A+B$	next
LW3	$\text{Reg}[\text{rt}] \leftarrow \text{Memory}$	spin
LW4		fetch
...		
SW0	$A \leftarrow \text{Reg}[\text{rs}]$	next
SW1	$B \leftarrow \text{sExt16}(\text{Imm})$	next
SW2	$MA \leftarrow A+B$	next
SW3	$\text{Memory} \leftarrow \text{Reg}[\text{rt}]$	spin
SW4		fetch

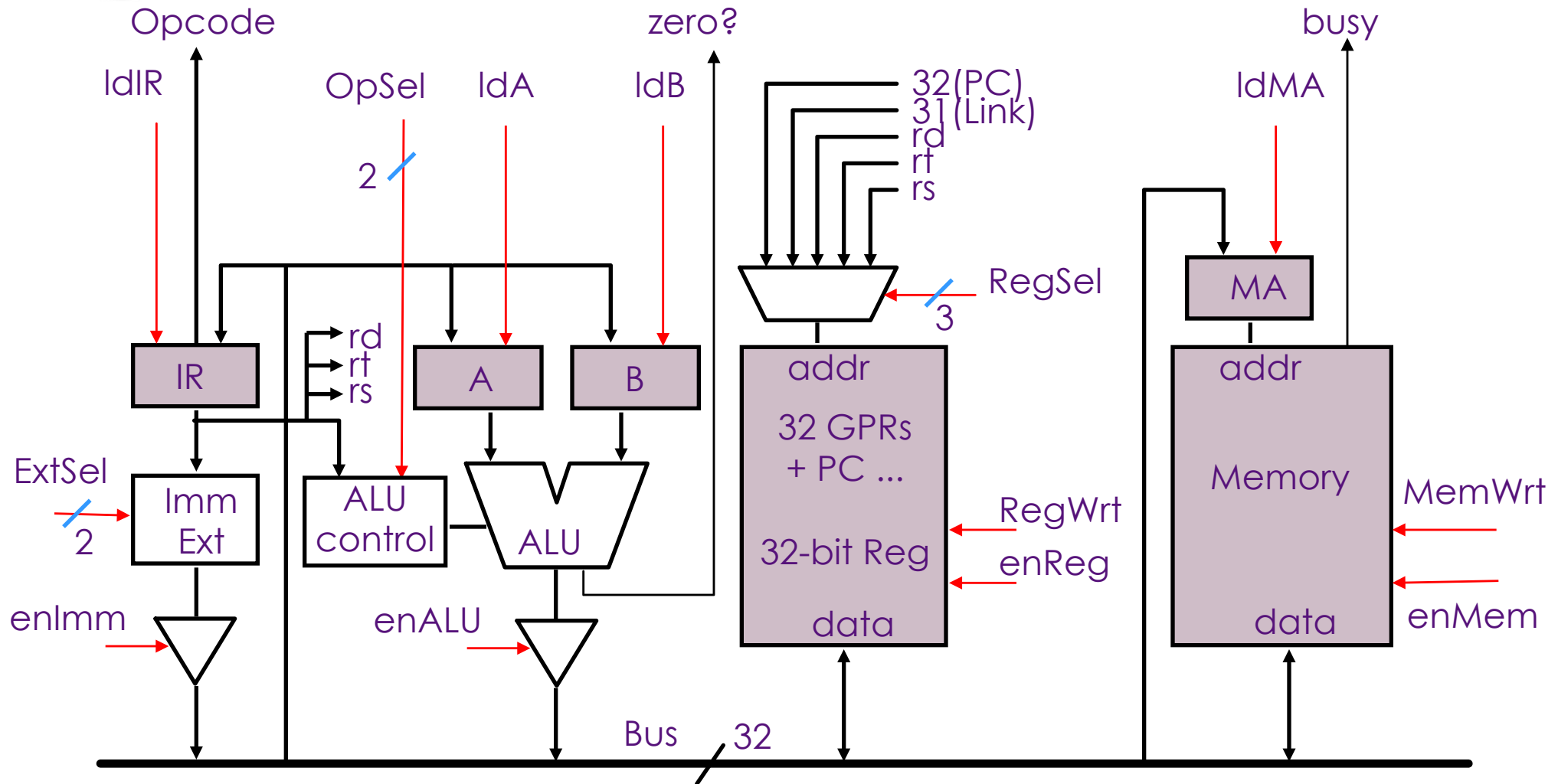


Jumps

State	Control Points	Next State
J0	$A \leftarrow PC$	next
J1	$B \leftarrow IR$	next
J2	$PC \leftarrow \text{JumpTarg}(A,B)$	fetch
JR0	$A \leftarrow \text{Reg}[rs]$	next
JR1	$PC \leftarrow A$	fetch
JAL0	$A \leftarrow PC$	next
JAL1	$\text{Reg}[31] \leftarrow A$	next
JAL2	$B \leftarrow IR$	next
JAL3	$PC \leftarrow \text{JumpTarg}(A,B)$	fetch
JALR0	$A \leftarrow PC$	next
JALR1	$B \leftarrow \text{Reg}[rs]$	next
JALR2	$\text{Reg}[31] \leftarrow A$	next
JALR3	$PC \leftarrow B$	fetch



Complex Instructions



$rd \leftarrow M[(rs)] \text{ op } (rt)$
 $M[(rd)] \leftarrow (rs) \text{ op } (rt)$
 $M[(rd)] \leftarrow M[(rs)] \text{ op } M[(rt)]$

Reg-Memory-src ALU op
Reg-Memory-dst ALU op
Mem-Mem ALU op



Mem-Mem ALU Instruction

$$M[(rd)] \leftarrow M[(rs)] \text{ op } M[(rt)]$$

State	Control Points	Next State
ALUMM ₀	$MA \leftarrow \text{Reg}[rs]$	next
ALUMM ₁	$A \leftarrow \text{Memory}$	spin
ALUMM ₂	$MA \leftarrow \text{Reg}[rt]$	next
ALUMM ₃	$B \leftarrow \text{Memory}$	spin
ALUMM ₄	$MA \leftarrow \text{Reg}[rd]$	next
ALUMM ₅	$\text{Memory} \leftarrow \text{func}(A,B)$	spin
ALUMM ₆		fetch

- With microcode, complex instructions do not change datapath and only require space for microprogram
- With hardwired control, complex instructions change datapath



Performance

Microcode requires multiple cycles per instruction

- $t_C > \max(t_{\text{reg-reg}}, t_{\text{ALU}}, t_{\mu\text{ROM}})$

Microcode requires multiple cycles per instruction

- Suppose $t_{\mu\text{ROM}} < 0.1 * t_{\text{RAM}}$
- Compare against single-cycle, hardwired control
- Good performance even when CPI=10

Microprogramming Advantages (1970's)

- ROMs were much faster than DRAMs
- Datapath, control were simpler for complex instruction sets
- Fixing bugs in the controller was easy
- ISA compatibility across models was easy



Decline of Microprogramming

Increasingly complex microcode

- Complex instruction sets led to subroutine, call stacks in microcode
- Fixing bugs difficult with read-only nature of ROMs

Evolving Technology

- VLSI made RAMs faster
- Microarchitectural innovations (pipelining, caches, buffers) made multi-cycle, register-to-register execution unattractive

Evolving Software

- Better compilers made complex instructions less important
- Most complex instructions are rarely used

Transition to RISC

- Build fast instruction cache
- Use software subroutines, not hardware subroutines
- Use simple ISA to enable hardwired implementations



Modern Microprogramming

Microprogramming is far from extinct

Important role in early microprocessors (1980s)

- Examples: Intel 386, 486

Assisting role in modern microprocessors

- Most instructions executed directly (hardwired control)
- Infrequent, complicated instructions invoke microcode engine
- Examples: AMD Athlon, Intel Core 2 Duo, IBM Power PC
- Patchable microcode common for post-fabrication bug fixes
- Example: Intel Pentiums load microcode patches at bootup



Acknowledgements

These slides contain material developed and copyright by

- Arvind (MIT)
- Krste Asanovic (MIT/UCB)
- Joel Emer (Intel/MIT)
- James Hoe (CMU)
- John Kubiatowicz (UCB)
- Alvin Lebeck (Duke)
- David Patterson (UCB)
- Daniel Sorin (Duke)